Deferred Substitution

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in Programming Languages: Application and Interpretation.

Cost of Substitution

What is the cost of this interp?

```
(interp (parse `{with {x 1}
                      {with {y 2}
                            {+ 100 {+ 99 {+ 98 ... {+ y x}}}}}}))
```

Cost of Substitution

What is the cost of this interp?

```
(interp (parse `{with {x 1}
                      {with {y 2}
                            {+ 100 {+ 99 {+ 98 ... {+ y x}}}}}}))
```

=>

```
(interp (parse `{with {x 1}
                      {with {y 2}
                            {+ 100 {+ 99 {+ 98 ... {+ y 1}}}}}}))
```

=>

```
(interp (parse `{with {x 1}
                      {with {y 2}
                            {+ 100 {+ 99 {+ 98 ... {+ 2 1}}}}}}))
```

With n variables, evaluation will take $O(n^2)$ time

Cost of Substitution

- Given n variables, we rewrite the body of the *with*s n times
- In addition, we compare the current variable to be substituted with ~ n – 1 other variables in the body
- With n variables, evaluation will take $O(n^2)$ time

Deferring Substitution

- We employ a space-time tradeoff to mitigate the cost of substitution
- We cache the substitution in a data structure called an environment
- An environment records names and their corresponding values

Deferring Substitution

- When we encounter a binding, we remember its value
- When we encounter a variable, we look up its value

Deferring Substitution

When a with is encountered, add the name value pair to a list rather than replace them.

```
(interp (parse `{with {x 1}
                      {with {y 2}
                            {+ 100 {+ 99 {+ 98 ... {+ y x}}}}}}))
```



```
(interp (parse `{with {y 2}
                      {+ 100 {+ 99 {+ 98 ... {+ y x}}}}}))
```




```
(interp (parse `{+ 100 {+ 99 {+ 98 ... {+ y x}}}}))
```




```
(interp (parse `{+ y x}}}))
```



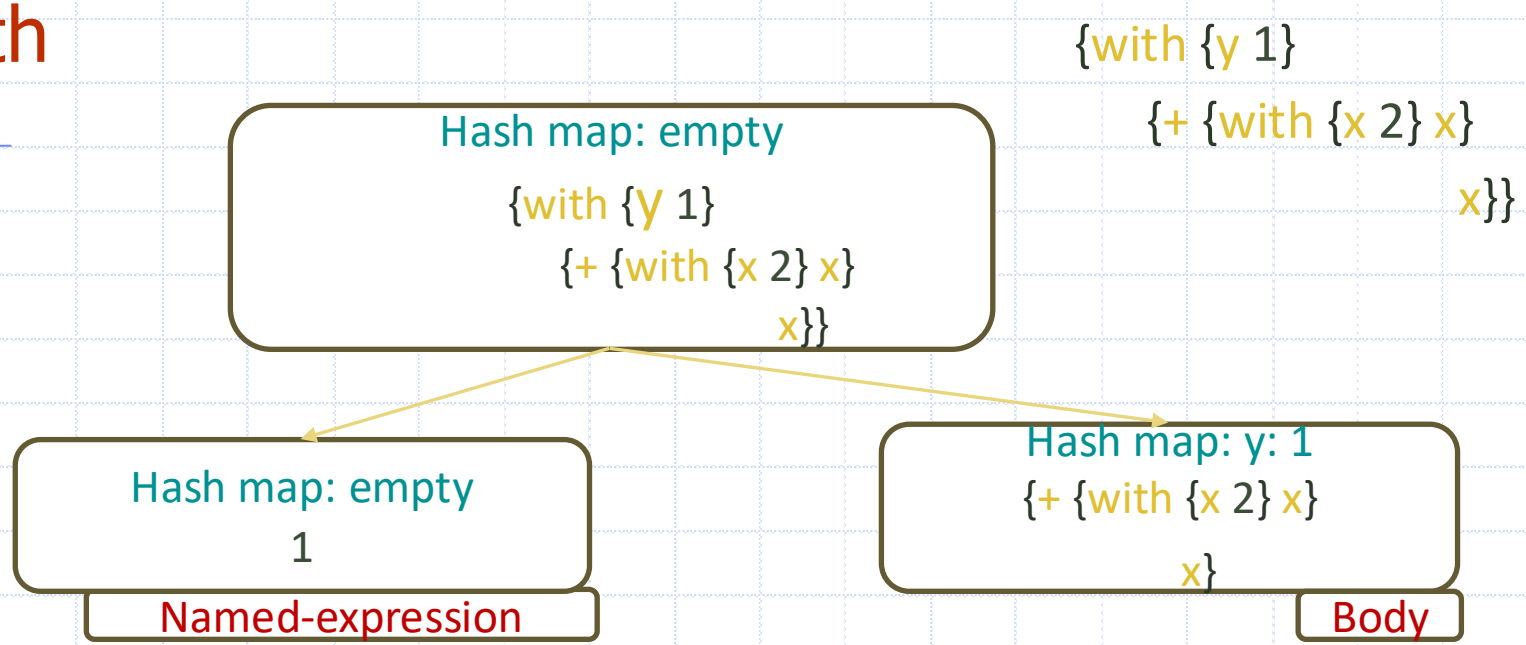
Interping with Deferred Substitution: multiple binding occurrences

Hash map: empty

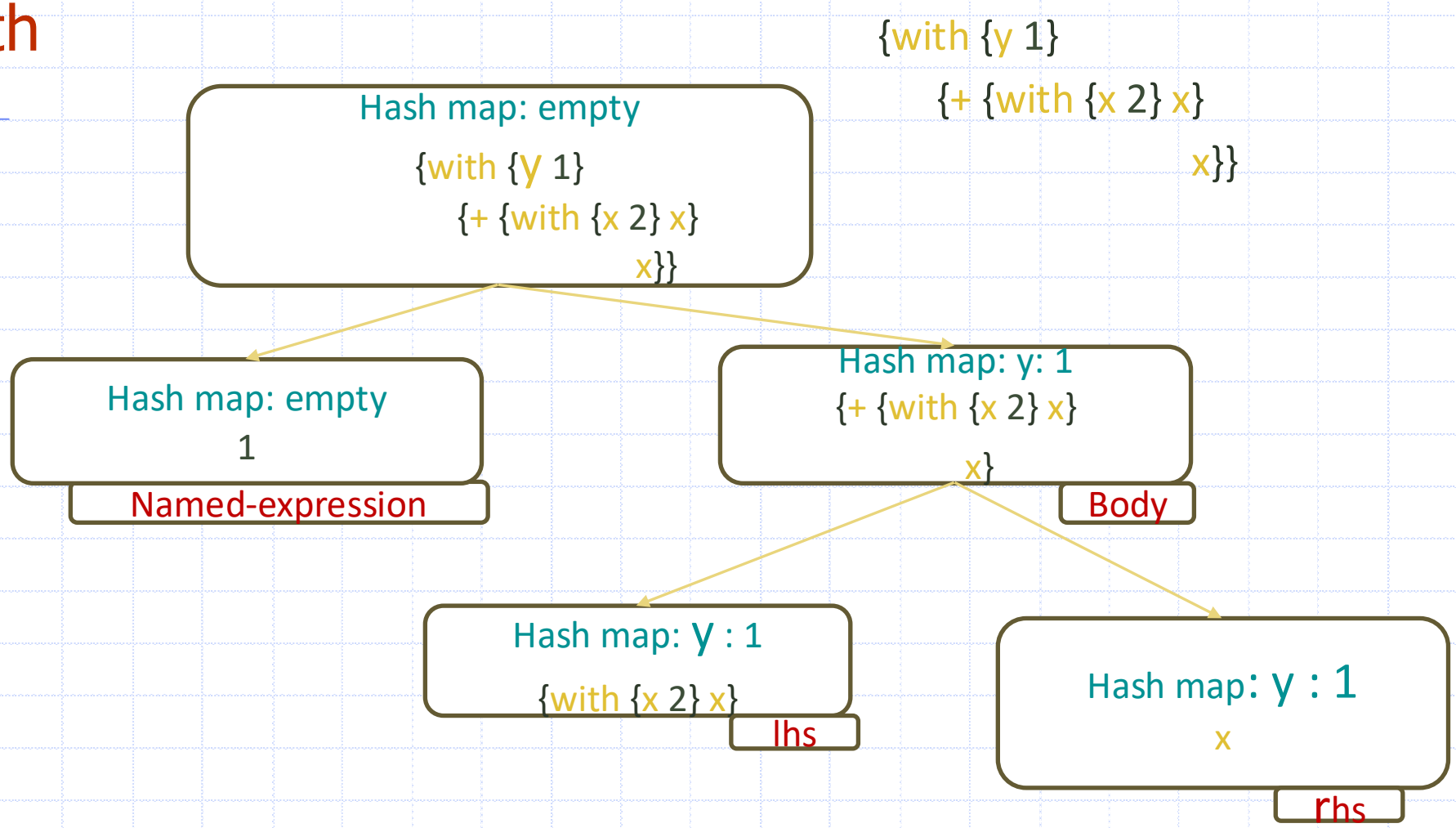
```
{with {y 1}
   {+ {with {x 2} x}
      x}}
```

```
{with {y 1}
   {+ {with {x 2} x}
      x}}
```

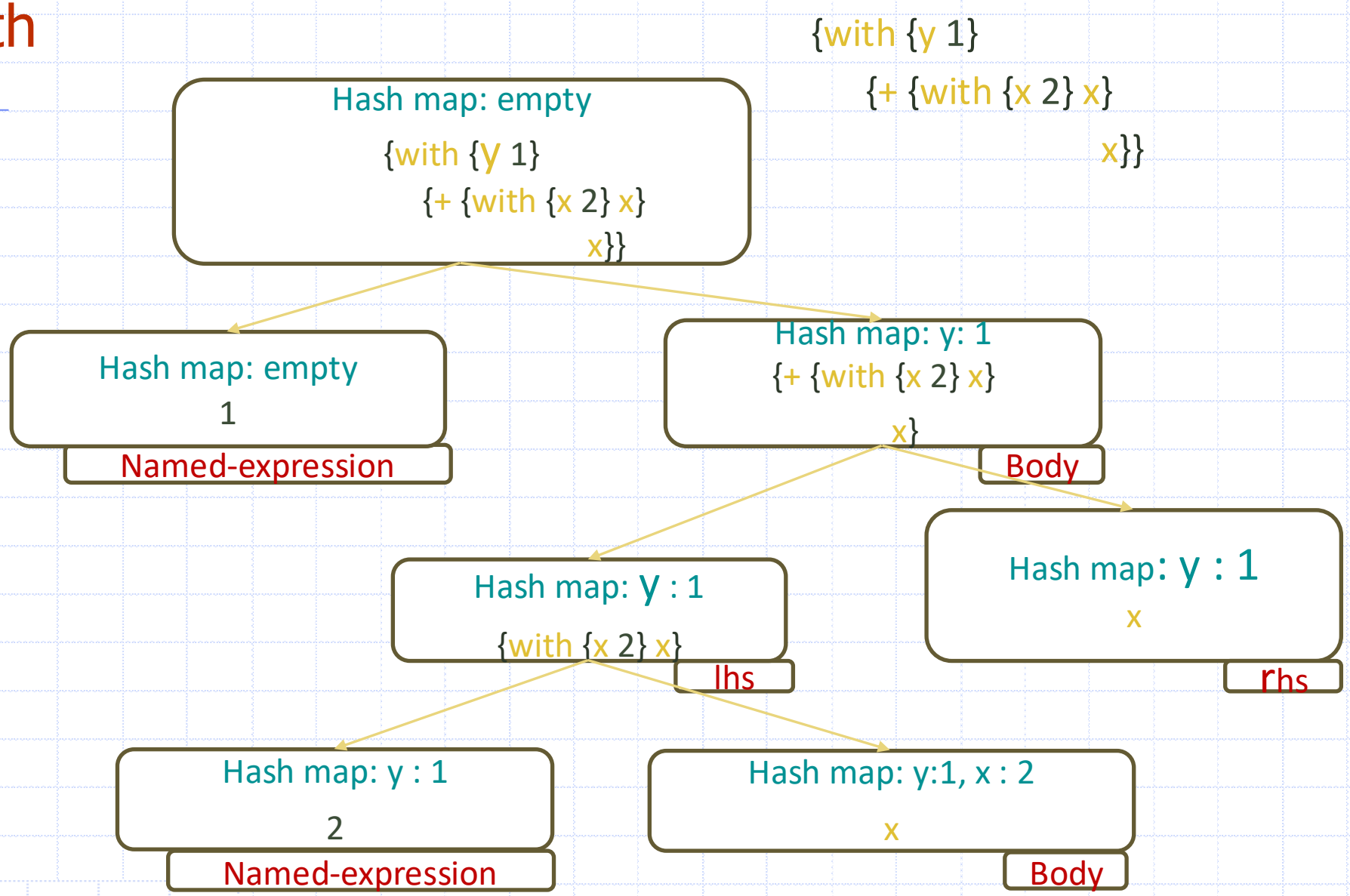
Interping with Deferred Substitution: multiple binding occurrences



Interping with Deferred Substitution: multiple binding occurrences



Interping with Deferred Substitution: multiple binding occurrences



Interping with Deferred Substitution: shadowed-ids

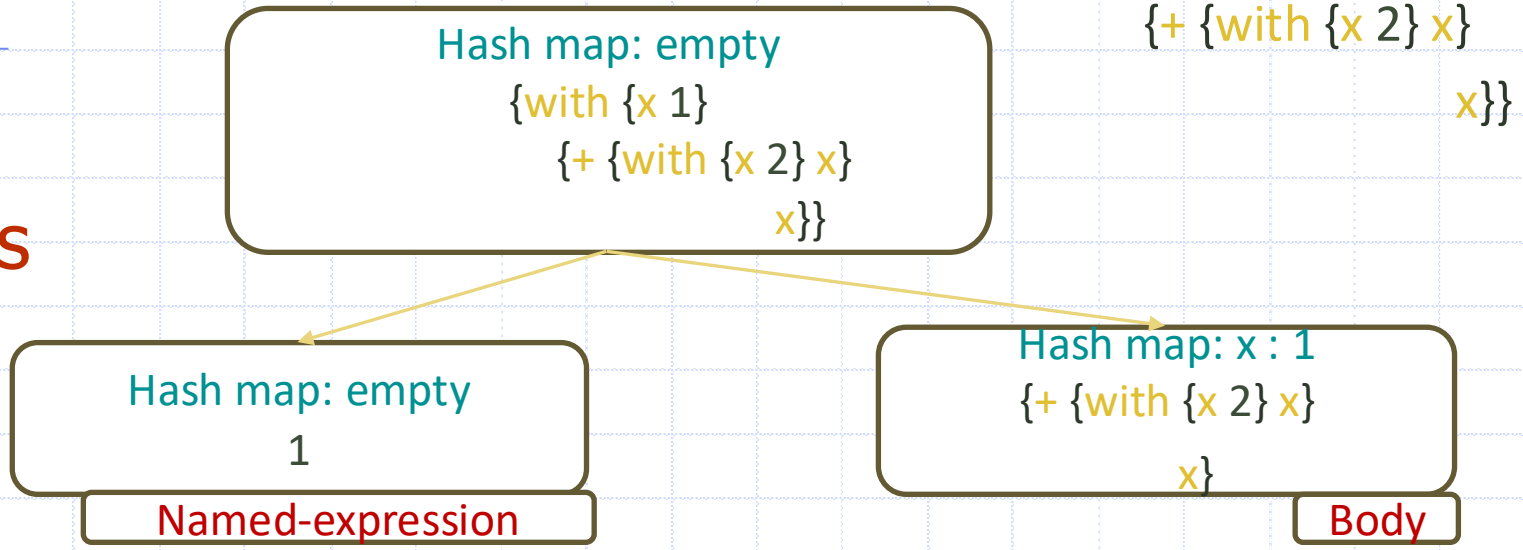
Hash map: empty

```
{with {x 1}
   {+ {with {x 2} x}
      x}}
```

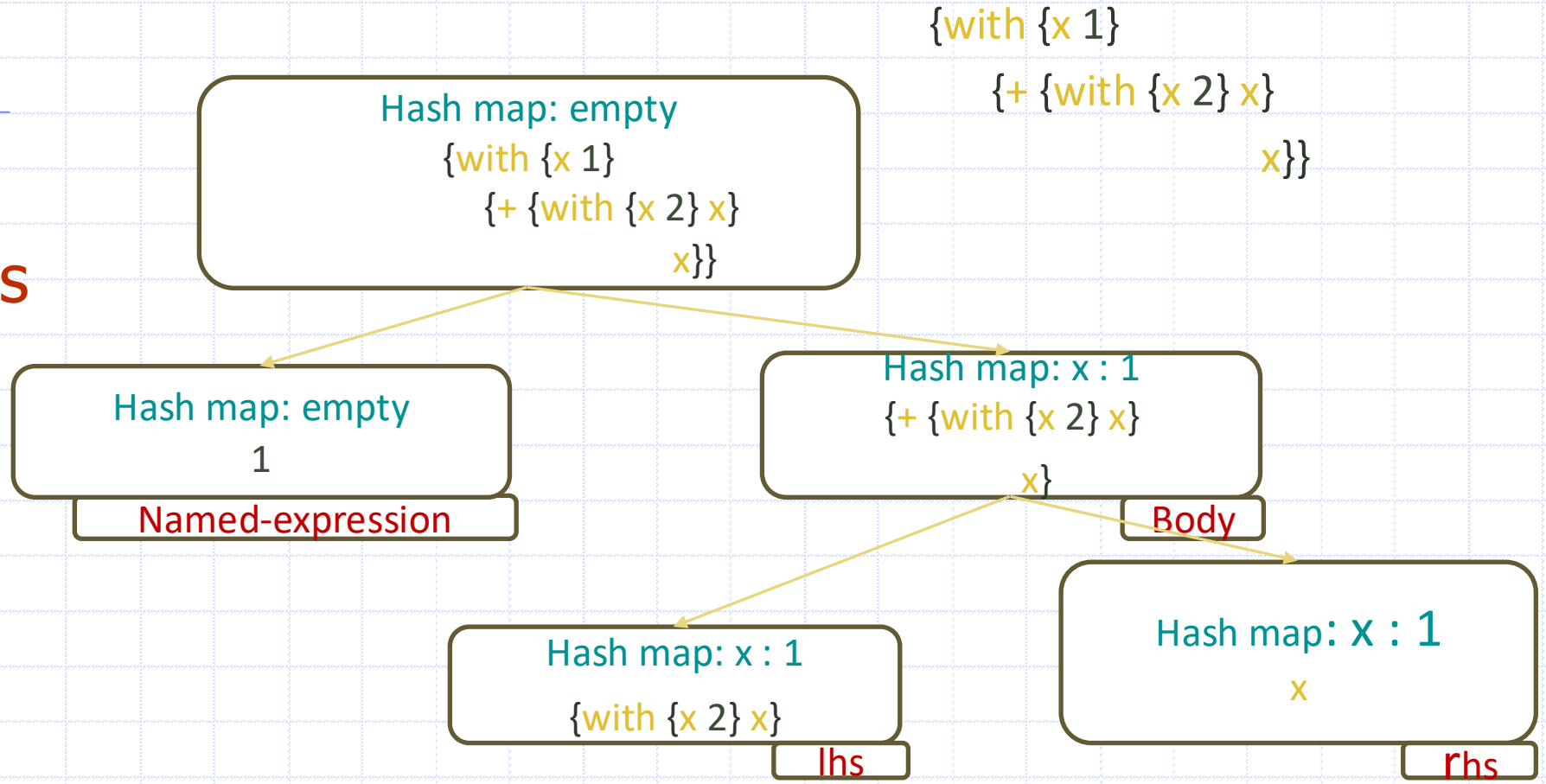
```
{with {x 1}
   {+ {with {x 2} x}
      x}}
```

Interping with Deferred Substitution: shadowed-ids

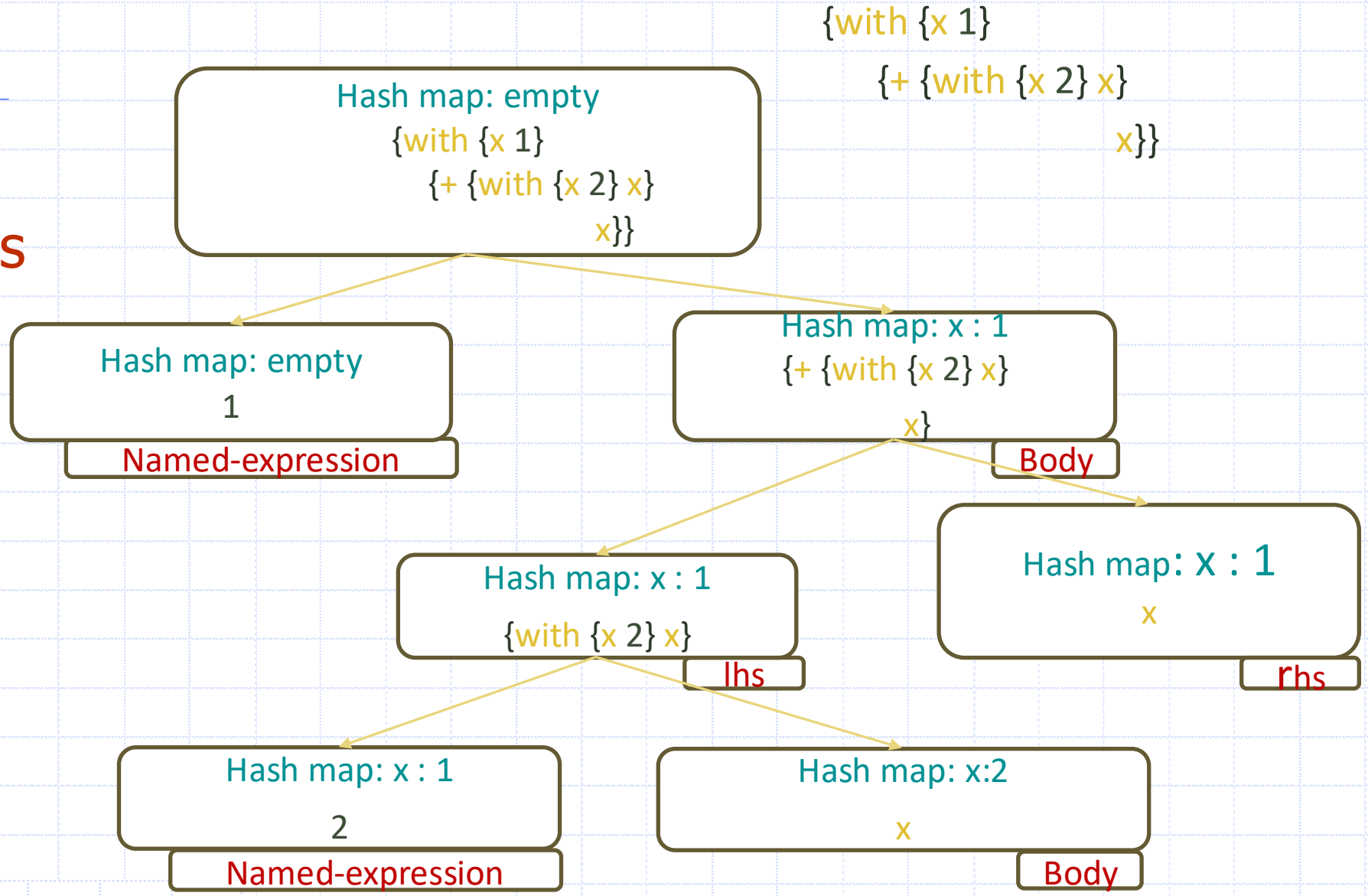
```
{with {x 1}
  {+ {with {x 2} x}
     x}}
```



Interping with Deferred Substitution: shadowed-ids



Interping with Deferred Substitution: shadowed-ids



Deferring Substitution

Example 2: Deferring substitution with the same identifier

```
(interp {with {x 1}
              {+ {with {x 2} x}
                 x}})
```



The hash table is empty at first

Deferring Substitution

Example 2: Deferring substitution with the same identifier

```
(interp {with {x 1}
                {+ {with {x 2} x}
                x}})
```






```
(interp {+ {with {x 2} x} x}))
```

The hash table is empty at first

After evaluating the outer **`with`**

Deferring Substitution

Example 2: Deferring substitution with the same identifier

```
(interp {with {x 1}
              {+ {with {x 2} x}
                 x}})
```






```
(interp {+ {with {x 2} x} x}))
```






```
(+ (interp {with {x 2} x}) (interp x))
```

The hash table is empty at first

After evaluating the first with

evaluating the rhs of plus

Deferring Substitution

Example 2: Deferring substitution with the same identifier

```
(interp {with {x 1}
              {+ {with {x 2} x}
                 x}})
```

x=1

```
(interp {+ {with {x 2} x} x}))
```

x=1

```
(+ (interp {with {x 2} x}) (interp x))
```

x=2

x=1

```
(+ (interp x) (interp x))
```

x=2

x=1

The hash table is empty at first

After evaluating the first with

evaluating the rhs of plus

Evaluating the lhs of plus.

The named-expression is evaluated with the original hash table

The body is evaluated with a new hash table.

Deferring Substitution

Example 2: Deferring substitution with the same identifier

```
(interp {with {x 1}
              {+ {with {x 2} x}
                 x}})
```

x=1

```
(interp {+ {with {x 2} x} x}))
```

x=1

```
(+ (interp {with {x 2} x}) (interp x))
```

x=2

x=1

```
(+ (interp x) (interp x))
```

x=2

x=1

```
(+ 2 1)
```

The hash table is empty at first

After evaluating the first with

evaluating the rhs of plus

Evaluating the lhs of plus.

The named-expression is evaluated with the original hash table

The body is evaluated with a new hash table.

Representing Deferred Substitution

Change

```
(interp : WAE -> Number))
```

to

```
(interp : (WAE (Hashof Symbol Number) -> Number))
```

Interp with Deferred Substitution

```
(interp {with {x 1}
              {with {y 2}
                    {+ 100 {+ 99 {+ 98 ... {+ y x}}}}} (hash empty))
=>
(interp {with {y 2}
              {+ 100 {+ 99 {+ 98 ... {+ y x}}}}}
        (hash (list (pair x 1))))

=>
(interp {+ 100 {+ 99 {+ 98 ... {+ y x}}}}
        (hash (list (pair x 1) (pair y 2))))

=>
(interp y (hash (list (pair x 1) (pair y 2))))
```

Interp with Deferred Substitution

```
(define (interp exp ds)
 (type-case WAE exp
  [(num n) n]
  [(var id) . . .]
  [(plus lhs rhs) (+ (interp lhs ds)
                     (interp rhs ds))]
  [(sub lhs rhs) (- (interp lhs ds)
                    (interp rhs ds))]
  [(with name named-exp body) . . .]))
```

Interp with Deferred Substitution

```
(define (interp exp ds)
 (type-case WAE exp
  [(num n) n]
  [(var id) . . .]
  [(plus lhs rhs) (+ (interp lhs ds)
                     (interp rhs ds))]
  [(sub lhs rhs) (- (interp lhs ds)
                    (interp rhs ds))]
  [(with name named-exp body) . . .(interp named-exp ds) . . .]))
```

Evaluate the named-expression with the existing set of symbols (environment)

Interp with Deferred Substitution

```
(define (interp exp ds)
 (type-case WAE exp
  [(num n) n]
  [(var id) . . .]
  [(plus lhs rhs) (+ (interp lhs ds)
                     (interp rhs ds))]
  [(sub lhs rhs) (- (interp lhs ds)
                    (interp rhs ds))]
  [(with name named-exp body) . . .
                              (hash-set ds name
                                        (interp named-exp ds)) . . .]))
```

Evaluate the body with the updated hash map that includes the name and the value of the named expression

Interp with Deferred Substitution

```
(define (interp exp ds)
 (type-case WAE exp
  [(num n) n]
  [(var id) . . .]
  [(plus lhs rhs) (+ (interp lhs ds)
                     (interp rhs ds))]
  [(sub lhs rhs) (- (interp lhs ds)
                    (interp rhs ds))]
  [(with name named-exp body) (interp body
```

Evaluate the body with the updated hash-map

```
(hash-set ds name
 (interp named-exp ds))]))
```

Interp with Deferred Substitution

```
(define (interp exp ds)
 (type-case WAE exp
  [(num n) n]
  [(var id) (lookup id ds)]
  [(plus lhs rhs) (+ (interp lhs ds)
                     (interp rhs ds))]
  [(sub lhs rhs) (- (interp lhs ds)
                    (interp rhs ds))]
  [(with name named-exp body) (interp body
                                      (hash-set ds name
                                       (interp named-exp ds)))]))
```

When a symbol is encountered, lookup the symbol value pair in the hash map

Lookup

```
(lookup : (Symbol (Hashof Symbol Number) -> Number))
(define (lookup id ds)
 (type-case (Optionof Number) (hash-ref ds id)
   [(none) (error 'not-found "not found")]
   [(some n) n]))
```

Function Calls

- Our list of functions are currently in the global environment (for now),
- while our expressions are in a local environment
- Binding occurrences in a local environment are not available in function bodies

Function calls

Java

```
public static void main(String[] args)
{
        {
                int y = 30;
                add(5);
        }
}
public static add(int x){
        return x + y;
}
```

WAE

```
{with {y 30}
      {add 5}}

{deffun {add x}
        {+ x y}}
```

Just like the y declared in a local scope in Java does not become available in a top-level environment, the same phenomenon is at work in our WAE

Function Calls

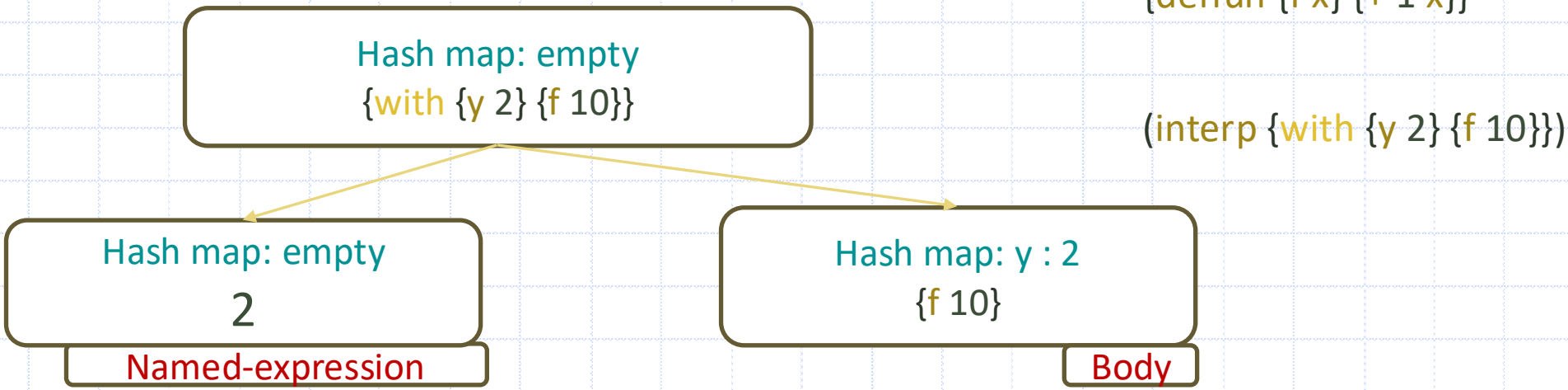
Hash map: empty
{with {y 2} {f 10}}

```
{deffun {f x} {+ 1 x}}

(interp {with {y 2} {f 10}})
```

Interping with Deferred Substitution: Function calls

Function Calls



{deffun {f x} {+ 1 x}}

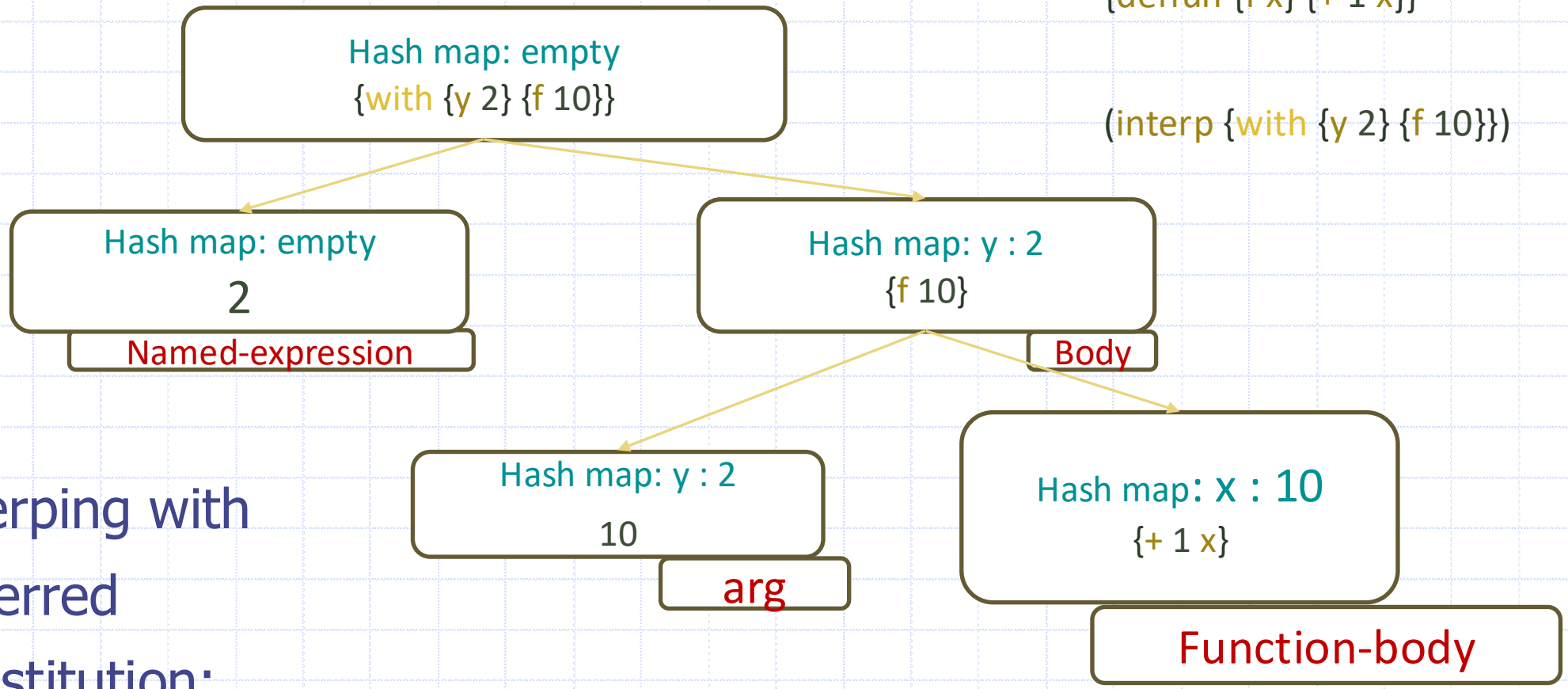
(interp {with {y 2} {f 10}})

Interping with Deferred Substitution: Function calls

Function Calls

{deffun {f x} {+ 1 x}}

(interp {with {y 2} {f 10}})

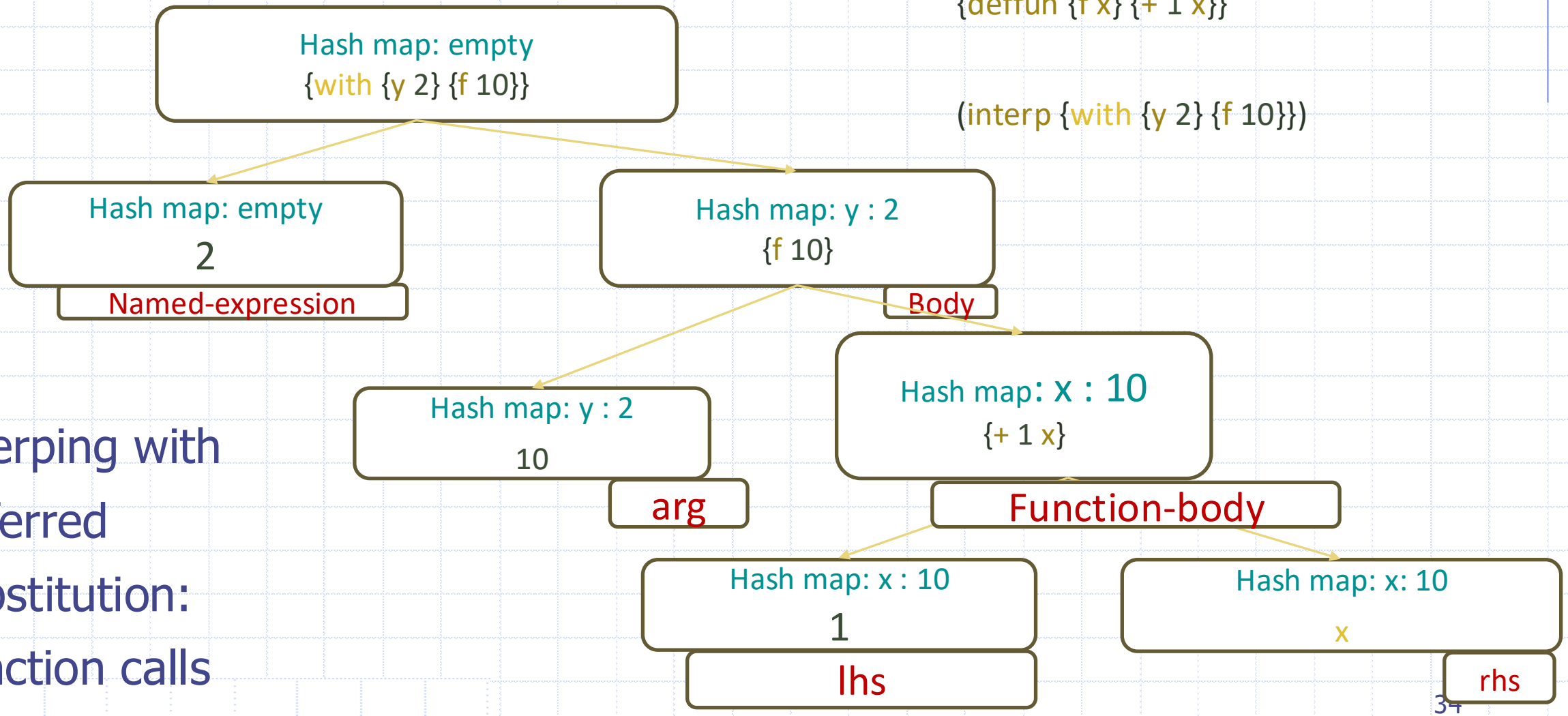


Interping with Deferred Substitution: Function calls

Function Calls

{deffun {f x} {+ 1 x}}

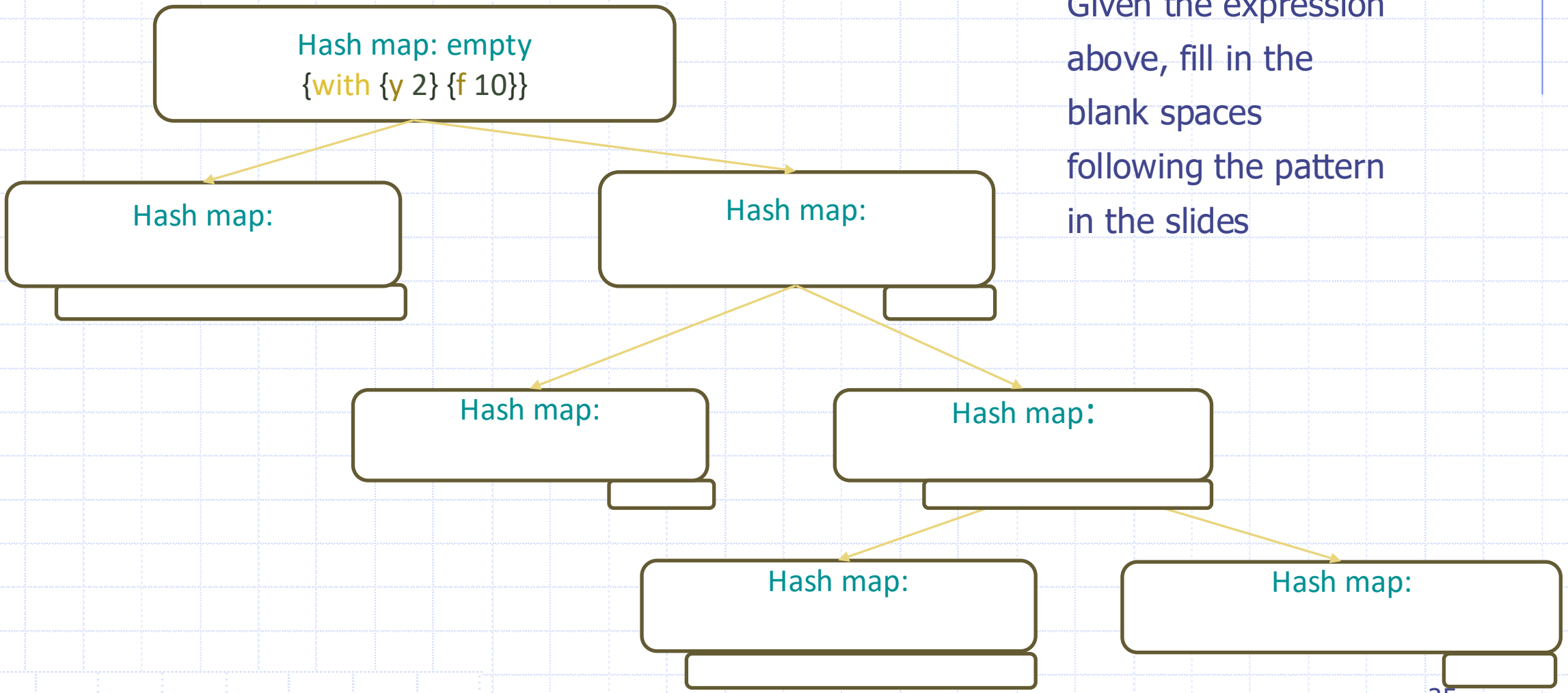
(interp {with {y 2} {f 10}})



Interping with Deferred Substitution: Function calls

Function Calls: Exercise

```
{deffun {f x} {+ y x}}
(interp {with {y 2} {f 10}})
```



Given the expression above, fill in the blank spaces following the pattern in the slides

Function Calls

```
{deffun {f x} {+ 1 x}}

(interp {with {y 2} {f 10}})
=>
(interp {f 10})
```





Function Calls

{deffun {f x} {+ 1 x}}



(interp {with {y 2} {f 10}})

=>



(interp {f 10})

=>



(interp {+ 1 x})

Interping the function body starts with an empty hash map

Our current interpreter's functions are from a global environment and none of the local variables are in the scope of the body of the function

Function Calls

What goes wrong if you extend the old substitution?

```
{deffun {f x} {+ y x}}
```



```
(interp {with {y 2} {f 10}})
```

Function Calls

What goes wrong with if you extend the old substitution?

{deffun {f x} {+ y x}}



(interp {with {y 2} {f 10}})

=>



(interp {f 10})

Function Calls

What goes wrong with if you extend the old substitution?

{deffun {f x} {+ y x}}



(interp {with {y 2} {f 10}})

=>

y=2

(interp {f 10})

=>

y=2, x=10

(interp {+ y x})

Function Calls

What goes wrong with if you extend the old substitution?

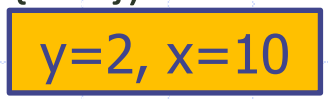
```
{deffun {f x} {+ y x}}
```



```
(interp {with {y 2} {f 10}})
=>
```




```
(interp {f 10})
=>
```




```
(interp {+ y x})
=> 12 ;; wrong!
```

Function Calls

What goes wrong with if you extend the old substitution?

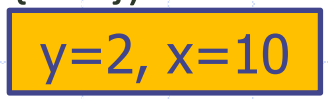
```
{deffun {f x} {+ y x}}
```



```
(interp {with {y 2} {f 10}})
=>
```




```
(interp {f 10})
=>
```




```
(interp {+ y x})
=> "free identifier"
```

Why not dynamic scope?

- I keep saying it's a poor design choice, but feel that I haven't really discussed what makes it so problematic (aside from a few examples).
- So, what is the issue?

Why not dynamic scope?

- Loss of Predictability and Readability: Under lexical (static) scoping, you can determine what a variable points to by reading the code around it. Under dynamic scoping, a function's behavior depends entirely on who called it and where it is invoked in the execution flow.

Why not dynamic scope?

- No Referential Transparency: Functions can no longer be treated as black boxes. An inner function might silently read or overwrite a variable belonging to an outer caller, violating expectations and breaking local reasoning.

Why not dynamic scope?

```
// A global or outer variable
var factor = 2;

function multiply(x) {
    return x * factor;
}

function processStandard(n) {
    return multiply(n);
}

function processPremium(n) {
    var factor = 10; // Redefines 'factor' locally
    return multiply(n);
}
```

Because multiply(5) returns 10 in one place and 50 in another, you can no longer reason about the function in isolation.

Why not dynamic scope?

- Opaque Dependencies and Accidental Collisions: Because variables are looked up dynamically up the stack, unrelated functions or libraries can accidentally step on each other's variable names if they happen to share an identifier. This creates hidden, implicit coupling across distant parts of a codebase.

Why not dynamic scope?

- Nightmare for Refactoring and Debugging: Changing or moving a function call can silently break child functions that rely on variables present in the old call stack. Debugging requires tracking the entire runtime history rather than examining local scope.

Why not dynamic scope?

- Compiler Limitations and Performance Costs: Compilers cannot resolve variable locations at compile time, preventing optimizations like inline expansion, register allocation, or dead-code elimination. Instead, the runtime must actively traverse stack frames to look up names.

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            . . .
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                                    . . .]
            . . .)]))
```

Lookup the function in funDefs using

name

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            . . .
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                    (define body (deffun-body func))
                                . . .]
             . . .)]))
```

Retrieve the body of the function

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            . . .
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                        (define body (deffun-body func))
                        (define param (deffun-param func))
                                    . . .]
             . . .)]))
```

Retrieve the parameter

name of the function

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            . . .
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                           (define body (deffun-body func))
                           (define param (deffun-param func))
                           (define value (interp arg ds funDefs))]
              . . .)]))
```

Evaluate the argument in the function call

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            . . .
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                    (define body (deffun-body func))
                    (define param (deffun-param func))
                    (define value (interp arg ds funDefs))]
                        . . . (make-hash (list (pair param value))). . .
)]))
```

Create a new hash-map with the param and the evaluated argument (value) pair

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
            ...
      [(app name arg)
            (local [(define func (lookup-function name funDefs))
                    (define body (deffun-body func))
                    (define param (deffun-param func))
                    (define value (interp arg ds funDefs))]
                   (interp body
                           (make-hash (list (pair param value)))
                           funDefs))]))
```

Evaluate the body of the function with the new hashmap as an input argument to interp

F1WAE Interpreter with Deferred Substitution

```
(define (interp a-f1wae ds funDefs)
 (type-case F1WAE a-f1wae
          . . .
      [(app name arg)
           (local [(define func (lookup-function name funDefs))
                   (define body (deffun-body func))
                   (define param (deffun-param func))
                   (define value (interp arg ds funDefs))]
                  (interp body
                          (make-hash (list (pair param value)))
                          funDefs))]))
```

Evaluate the body of the function with the new hashmap as an input argument to interp

Question? Why don't we append the created hash to ds and pass that to the invocation of interp on the body?